Creating a Sequence Diagram

Step-by-Step Guide, Uses, and Challenges

Steps for Building a Sequence Diagram

1. Set the context
2. Identify which objects and actors will participate
3. Set the lifeline for each object/actor
4. Lay out the messages from top to bottom based on the order in which they are sent
5. Add the focus of control (activation box) for each object's or actor's lifeline
6. Validate the sequence diagram

Creating a Sequence Diagram (Detailed Guide)

Goal

A sequence diagram is created during the design phase to illustrate how objects/actors interact over time via messages.

1. **Identify the Scenario:** Select the specific scenario or use case to model.
2. **List the Participants:** Identify actors/objects involved (users, systems, external entities).
3. **Define Lifelines:** Draw a vertical dashed lifeline for each participant.
4. **Arrange Lifelines:** Place lifelines left-to-right based on involvement/order of access.
5. **Add Activation Bars:** Add activation (focus of control) to show processing duration.
6. **Draw Messages:** Use arrows to represent communications (synchronous, asynchronous, self).
7. **Include Return Messages:** Use dashed arrows for responses/returns when needed.
8. **Indicate Timing and Order:** Number messages if required; preserve top-to-bottom order.
9. **Include Conditions and Loops:** Represent conditions and repetition using control constructs.
10. **Consider Parallel Execution:** Show parallel activities when concurrency exists.
11. **Review and Refine:** Check clarity, correctness, and completeness.

12. **Add Annotations and Comments:** Provide clarifications and context where needed.
13. **Document Assumptions and Constraints:** Record anything required for understanding/validity.
14. **Tools:** Use UML tools/diagramming software for clean, professional diagrams.

Exam Tip

Always ensure the message order matches the use case normal scenario and the correct "owner class" receives each message.

Step 1: Set the Context

- Select a **use case**
- Decide the **initiating actor**

Step 2: Identify Participants and Messages

2(a) List Candidate Objects

- Use case controller class
- Domain classes
- Database table classes
- Display screens or reports

2(b) List Candidate Messages (Message Analysis Table)

1. Examine each step in the normal scenario of the use case.
2. For each step:
   - Identify step number
   - Determine needed messages
   - Decide which class holds data or performs the action
3. Ensure messages are in the same order as the normal scenario.

2(c) Place Actors/Objects Across the Top

Typical left-to-right order:

- Actor

- Primary display screen class
- Primary use case controller class
- Domain classes (in order of access)
- Other display screens (in order of access)

Step 3: Set Lifelines

Lifelines

Add lifelines for each actor/object to represent their existence over time.

Step 4: Lay Out Messages

1. Draw each message arrow with the message name pointing to the owner class.
2. Decide which object/actor initiates the message and connect it to its lifeline.
3. Add needed return messages.
4. Add parameters and control information when needed.

Step 5: Add Focus of Control

Focus of Control (Activation Box)

Add activation boxes to show when an actor/object is active (executing, sending, or receiving messages).

Step 6: Validate the Sequence Diagram

- Matches the use case normal scenario
- Correct objects/actors included
- Correct message order (top-to-bottom)
- Correct destination (owner class)
- Needed returns, parameters, conditions included
- Activations consistent with message flow

Use Cases of Sequence Diagrams

Where Sequence Diagrams Are Useful

- **System Behavior Visualization:** Shows message and event flow over time.
- **Software Design and Architecture:** Helps plan how components work together.
- **Communication and Collaboration:** Improves team and stakeholder understanding.
- **Requirements Clarification:** Ensures a shared understanding of interactions.
- **Debugging and Troubleshooting:** Helps track order/timing to find issues.

Challenges of Using Sequence Diagrams

Common Challenges

- **Complexity and Size:** Large systems produce large, hard-to-read diagrams.
- **Abstraction Level:** Too much detail overwhelms; too little becomes unclear.
- **Dynamic Nature:** Diagrams must be updated as systems evolve.
- **Ambiguity in Messages:** Unclear message meaning can confuse stakeholders.
- **Concurrency and Parallelism:** Parallel interactions are hard to visualize cleanly.
- **Real-Time Constraints:** Precise timing constraints may require extra documentation.

Good Practice

For complex logic, draw multiple diagrams (one per scenario/alternative path) rather than forcing everything into one.